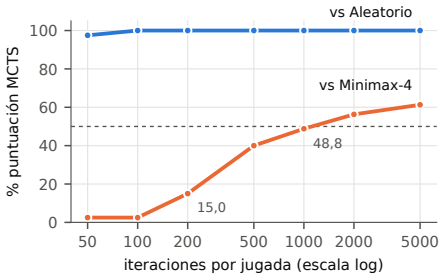
Monte Carlo Tree Search sobre Conecta 4 — Informe

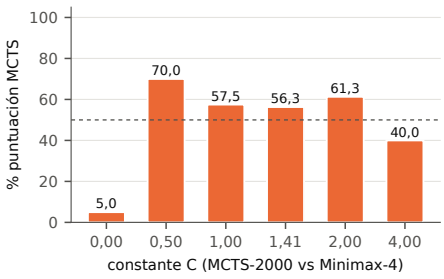
Alumno/a 1: _____ Alumno/a 2: _____

Datos: `referencia_s42.csv` · semilla 42 · 40 partidas por punto Máquina: Intel Core i9-14900K · 32 GB DDR5 · Windows 11

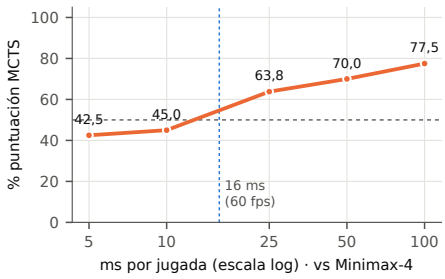
Todas las cifras salen de `dotnet run -- referencia 40 42`. Sólo se ha modificado `MotorMcts.cs`. Las tres pruebas de humo pasan: `traza` elige la columna 3 en los seis presupuestos (28,0 % de visitas con 25 simulaciones y 94,7 % con 10.000); `duelo 40 2000 4 42` da 56,3 % (22-17-1), y MCTS-500 puntúa 100 % contra el aleatorio. E1, E2 y E3 coinciden fila a fila con la tabla de referencia del enunciado. Con 40 partidas, el error típico cerca del 50 % es de unos ±8 puntos, así que no interpretamos diferencias de menos de ~15.



E1 · % de MCTS frente al presupuesto



E3 · % de MCTS-2000 frente a C



E4 · % de MCTS frente a ms por jugada

E1 · Presupuesto *¿Cuándo deja de ser un generador de números aleatorios? ¿Por qué se aplana?*

Contra el aleatorio, MCTS gana desde el primer punto (97,5 % con 50 iteraciones y 100 % desde 100). Ese rival no distingue un algoritmo malo de uno bueno. La prueba real es Minimax-4 con heurística: con 50 y 100 iteraciones MCTS saca un **2,5 %**, es decir, juega a ciegas. Con 200 llega al 15 % y con 500 al 40 %. **Con 1.000 iteraciones alcanza el empate técnico (48,8 %, 19-20-1)** y desde 2.000 gana más de lo que pierde (56,3 %). Por tanto, el umbral está entre **500 y 1.000 simulaciones**, que en esta máquina son unos 2–4 ms por jugada.

La curva se aplana (56,3 → 61,3 % al pasar de 2.000 a 5.000, por debajo del ruido) por dos razones. La primera es estadística: el error de la media de cada hijo cae como $1/\sqrt{n}$, así que duplicar el presupuesto sólo lo reduce un 30 %, y con 2.000 iteraciones la jugada buena ya se lleva la mayoría de las visitas. Las iteraciones extra confirman lo que ya se sabía. La segunda es de sesgo: los *rollouts* son uniformes, y un jugador aleatorio no sabe tapar una amenaza. Por eso, el valor al que converge cada nodo es el de «jugar al azar desde aquí», no el valor minimax. Más simulaciones afinan esa estimación sesgada, pero no corrigen el sesgo.

E2 · Heurística contra profundidad *¿Le sirve al ciego pasar de profundidad 4 a 6?*

Rival (MCTS-2000)	prof. 2	prof. 4	prof. 6
Minimax con heurística	58,8 %	56,3 %	23,8 %
Minimax ciego	96,3 %	92,5 %	92,5 %

No le sirve de nada. Contra el ciego, MCTS saca **92,5 %** a profundidad 4 y **92,5 %** a profundidad 6 (36-2-2 en ambos casos): mejora 0 puntos. Contra el rival con heurística, el mismo cambio de profundidad hunde a MCTS de 56,3 % a 23,8 %, lo que supone 32,5 puntos de mejora para el rival. El minimax ciego sólo distingue «gano / pierdo / nada» dentro de su horizonte. Casi todas sus hojas valen 0, así que con 6 capas ve más hojas igual de mudas y sigue eligiendo prácticamente al azar entre las jugadas que no pierden en seguida. **Buscar más hondo multiplica la calidad de la evaluación, no la sustituye**: la profundidad amplifica una buena heurística y no aporta nada si no la hay. MCTS no necesita heurística porque la sustituye por los rollouts, y por eso le gana al ciego a cualquier profundidad.

E3 · Constante de exploración *¿Qué le pasa con C = 0? ¿Y con C = 4?*

Con **C = 0** MCTS cae al **5,0 %** (2-38-0). Es el fallo de la **explotación pura (avaricia, convergencia prematura)**: cada hijo se prueba una vez porque los no visitados valen ∞, y a partir de ahí se baja siempre por el de mejor media. Una jugada que tuvo suerte en su primer rollout se queda todo el presupuesto, y las demás no se vuelven a mirar aunque sean mejores. Con **C = 4** baja al **40,0 %**. Es el fallo contrario, la **sobreexploración**: el término de exploración domina a la media, las visitas se reparten casi por igual entre jugadas buenas y malas, y el árbol no profundiza. Se parece a un Monte Carlo plano que gasta simulaciones en jugadas que ya sabe malas. Entre 0,5 y 2 todo queda entre 56 y 70 %. El 70 % de C = 0,5 es el mejor, pero su diferencia con $\sqrt{2}$ (56,3 %) no supera el ruido de 40 partidas. La curva tiene la forma esperada: se hunde en los dos extremos y es plana en medio.

E4 · Presupuesto de tiempo *Con 16 ms (un frame a 60 fps), ¿lo pondríamos en un juego?*

ms por jugada	5	10	25	50	100
Iteraciones que caben (`calibrar 42`)	586	1.511	3.277	5.696	11.822
% contra Minimax-4 (E4)	42,5	45,0	63,8	70,0	77,5

La curva crece de forma monótona (42,5 → 77,5 %), como se espera de un algoritmo *anytime*. Interpolando la calibración entre 10 ms (1.511) y 25 ms (3.277), en **16 ms caben ≈ 2.200 iteraciones** en nuestra máquina, medidas en la posición inicial, que es la más cara. La calibración incluida en el propio CSV da prácticamente lo mismo (1.521 y 3.295, ≈ 2.230). En E1, ese presupuesto corresponde a la fila de **2.000 iteraciones: 56,3 % contra Minimax-4** y 100 % contra el aleatorio. En E4 directamente, 16 ms queda entre el 45,0 % (10 ms) y el 63,8 % (25 ms).

Sí, lo pondríamos, con matices. En 16 ms MCTS ya gana a un alfa-beta de profundidad 4 con una heurística hecha a mano. Además, es *anytime*: si el frame se acorta, devuelve la mejor jugada que tenga. Pero en un juego real la IA no dispone del frame entero, porque el render y la lógica también lo usan. Lo razonable es repartir la búsqueda entre varios frames (el rival humano tarda segundos en mover) o lanzarla en un hilo aparte. Con 100 ms, que son 6 frames, sube al 77,5 %.

Observaciones sobre nuestros datos:

1. **E4 rinde más que E1 a iteraciones parecidas.** 25 ms (≈ 3.300 iteraciones) da 63,8 %, igual o más que E1 con 5.000 iteraciones (61,3 %). La calibración se mide en la posición inicial; a medida que el tablero se llena, los rollouts son más cortos y en el mismo tiempo caben más iteraciones.
2. **Nuestra calibración es más baja que la de la máquina de referencia** (586 frente a ≈ 1.100 en 5 ms), a pesar de ser un i9-14900K. Una explicación plausible es que `dotnet run` compila en Debug, pero no lo hemos verificado.
3. **El corte por tiempo se pasa alrededor de 1 ms** (5 ms → 6,0 ms reales, 10 → 11,0). La causa es que el TODO 7 compara `ElapsedMilliseconds` (milisegundos enteros) con `>`. Usar `Elapsed.TotalMilliseconds >= msMaximos` sería más estricto, pero lo hemos dejado así para que el CSV entregado corresponda al código entregado.

Como las cifras de E4 dependen del reloj, las tomamos como rangos y no como valores exactos.

> **Conclusión.** Usaríamos **MCTS** cuando no hay una buena función de evaluación, el factor de ramificación es alto o hace falta una respuesta *anytime* con un presupuesto variable. E2 lo demuestra: sin heurística, el minimax no mejora con la profundidad y MCTS le gana con un 92,5 %. Usaríamos **alfa-beta** cuando existe una heurística fiable y el juego es táctico y de ramificación baja, con trampas cortas que los rollouts aleatorios no ven. Con heurística y profundidad 6, alfa-beta deja a MCTS-2000 en un 23,8 %. En resumen: el alfa-beta vale lo que vale su heurística, y MCTS vale lo que valen sus simulaciones.

Opcional O4: las tres gráficas se generan desde `referencia_s42.csv` con `Entrega/graficas.py` (Python + matplotlib).